

Name: M.Asaad

Roll No: L1F22BSCS0568

Phase-2

COMPILER CONSTRUCTION

Language Name: PakLang

OverView:

PakLang is a simplified Mini C++-like programming language designed for educational purposes.

Its goal is to help beginners understand lexical analysis through meaningful and easy-to-remember keywords derived from Urdu and simple English concepts.

PakLang supports:

- variables (numbers & decimals)
- keywords for decision control (agar, warna)
- loop control (dobarah, chalo, ruko)
- string printing (likho)
- boolean values (sach, jhoot)
- numeric operations
- punctuation controls similar to C-like languages

PakLang is intentionally minimalistic and readable.

Keywords:

Keyword	Meaning / Equivalent (in C/C++)	Usage Example
agar	if	agar (x > 10)
warna	else	warna { ... }
jab	while	jab (i < 5)
dobarah	for	dobarah (i = 0; i < n; i = i + 1)
wapis	return	wapis x;
karo	do	karo { ... } jab (cond);
chalo	continue	chalo;
ruko	break	ruko;

Keyword	Meaning / Equivalent (in C/C++)	Usage Example
sach	true	bol flag = sach;
jhoot	false	bol flag = jhoot;
likho	print / output	likho("Hello");
riyazi	int	riyazi a = 10;
angerazi	float	angerazi b = 3.14;
poucho	input	poucho x;
HanYahNah	bool	HanYahNah ok = sach;

Reason of choosing these keywords:

The keywords in **PakLang** were carefully selected to make programming feel familiar and intuitive for beginners, especially those who think and communicate in Urdu or Hindi. Each keyword is meaningful, short, and easy to pronounce — helping learners quickly connect the logic of programming with everyday language.

Here's the reasoning behind some of the core choices:

- **agar (if)** → In Urdu/Hindi, *agar* means “if” or “in case.” It naturally expresses a condition, just like the English *if*, making it instantly understandable when writing conditional statements.
Example: agar ($x > 10$) reads like “if x is greater than 10.”
- **warna (else)** → Commonly used in everyday speech meaning “otherwise” or “else.” It pairs naturally with *agar*, forming a clear and conversational logic structure — *agar... warna...* (“if... otherwise...”)
- **jab (while)** → Literally means “when” or “while.” It fits perfectly for loops that continue as long as a condition is true.
- **dobarah (for)** → Means “again” or “repeat.” Ideal for representing *for loops*, where an action repeats multiple times.
- **wapis (return)** → The word *wapis* means “to return” or “to give back,” exactly matching the concept of returning a value from a function.
- **karo (do)** → A simple word meaning “do” or “perform.” It captures the directness of executing an action, as in *do-while* loops.
- **chalo (continue)** → Means “go on” or “keep going.” Perfectly fits the *continue* keyword, telling the program to move on to the next iteration.
- **ruko (break)** → Means “stop” or “pause.” This makes sense for breaking out of a loop or halting execution.
- **sach (true)** and **jhoot (false)** → Everyday words meaning “truth” and “falsehood.” These feel much more relatable than *true* and *false*, making logical expressions easier to grasp.
- **likho (print/output)** → Literally means “write.” Since printing or displaying output involves writing something on the screen, it's a natural, intuitive fit.

- **riyazi (integer)** → Comes from *riyaziyat* (mathematics). It represents numeric values (integers), directly connecting to basic arithmetic.
- **angerazi (float)** → A lighthearted, easy word representing decimal or non-integer numbers — keeping the tone friendly and simple.
- **poucho (input)** → Means “ask” or “inquire.” This aligns perfectly with taking input from the user — the program “asks” for information.
- **HanYahNah (boolean type)** → Translates to “Yes or No,” directly representing boolean logic (true/false). It’s descriptive and easy for beginners to relate to.

Operators:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
==	Equality Comparison

Punctuation:

Symbol	Usage
()	Grouping
{ }	Code blocks
,	Separator
;	Statement terminator

Date: _____

Keywords.

- (1) agar $\rightarrow$ if
- (2) warna $\rightarrow$ else
- (3) dobarah $\rightarrow$ loop
- (4) likho $\rightarrow$ Print
- (5) vijazi $\rightarrow$ integer data type

Operators.

- (1) + $\rightarrow$ Addition
- (2) - $\rightarrow$ Subtract
- (3) = $\rightarrow$ Assignment

Punctuation.

- (1) { } $\rightarrow$ scop
- (2) ; $\rightarrow$ statement terminator.

CFG (Content free grammar)

$G \rightarrow (V, T, P, S)$

V (Non-Terminal)

Program, start, G, F, D, A, C, L, P, E etc

T (Terminal)

agar, warna, =, +, {, }, (,), ; etc

S (start symbol)

Program

48

GRAMMER RULES

Program $\rightarrow$ SLT
 SLT $\rightarrow$ SLT ST | ST
 ST $\rightarrow$ D | A | C | L | P
 D $\rightarrow$ rigazi i;
 A $\rightarrow$ i = E
 C $\rightarrow$ agar (E) { SLT } warna { SLT }
 L $\rightarrow$ abarati (E) { SLT }
 P $\rightarrow$ likho (i);
 E $\rightarrow$ i
 E $\rightarrow$ number
 E $\rightarrow$ E + E | E

PRODUCTION RULES.

Program $\rightarrow$ SLT
 SLT $\rightarrow$ SLT ST | ST
 ST $\rightarrow$ D | A | C
 D $\rightarrow$ rigazi i;
 A $\rightarrow$ i = E
 E $\rightarrow$ number
 P $\rightarrow$ likho (i)
 C $\rightarrow$ agar (i == Num) { SLT }

First and Follow

Date: _____

FIRST & FOLLOW

RULES	FIRST	FOLLOW
Program $\rightarrow$ SLT	Riyazi, i, agar dobarah, likho	{ \$ }
SLT $\rightarrow$ SLT ST ST	Riyazi, i, agar dobarah, likho	{ \$, Riyazi, likho, agar, dobarah, i }
ST $\rightarrow$ D C A L P	Riyazi, i, agar dobarah, likho	{ \$, Riyazi, likho, i, dobarah, agar }
D $\rightarrow$ Type		
D $\rightarrow$ Riyazi i	Riyazi	
A $\rightarrow$ i = E	i	{ \$, Riyazi, likho, dobarah, i, agar }
C $\rightarrow$ agar (E) { SLT }	agar	{ \$, Riyazi, likho, agar, dobarah }
L $\rightarrow$ dobarah (E)	dobarah	{ \$, Riyazi, agar, dobarah, i, likho }
P $\rightarrow$ likho (i)	likho	{ \$, Riyazi, agar, dobarah, likho, i }
E $\rightarrow$ E == Num	i, Num, ε	{ \$, i, } }
E $\rightarrow$ Num	i, Num, ε	{ \$, i, } }
E $\rightarrow$ E + E ε	i, Num, ε	{ \$, i, } }
E $\rightarrow$ i		

Ambiguity Check

Date: _____

Ambiguity Check

Yes, the grammar is ambiguous due to the rule

$$E \rightarrow E + E \mid \epsilon$$

Example

$$a + b + c$$

This can be parsed as two way

$$\rightarrow (a + b) + c$$

$$\rightarrow a + (b + c)$$

This is the possible solution.

$$E \rightarrow (E)$$

$$E \rightarrow E + E \mid \epsilon$$

PARSE TREE CONSTRUCTION

Riyazi x;

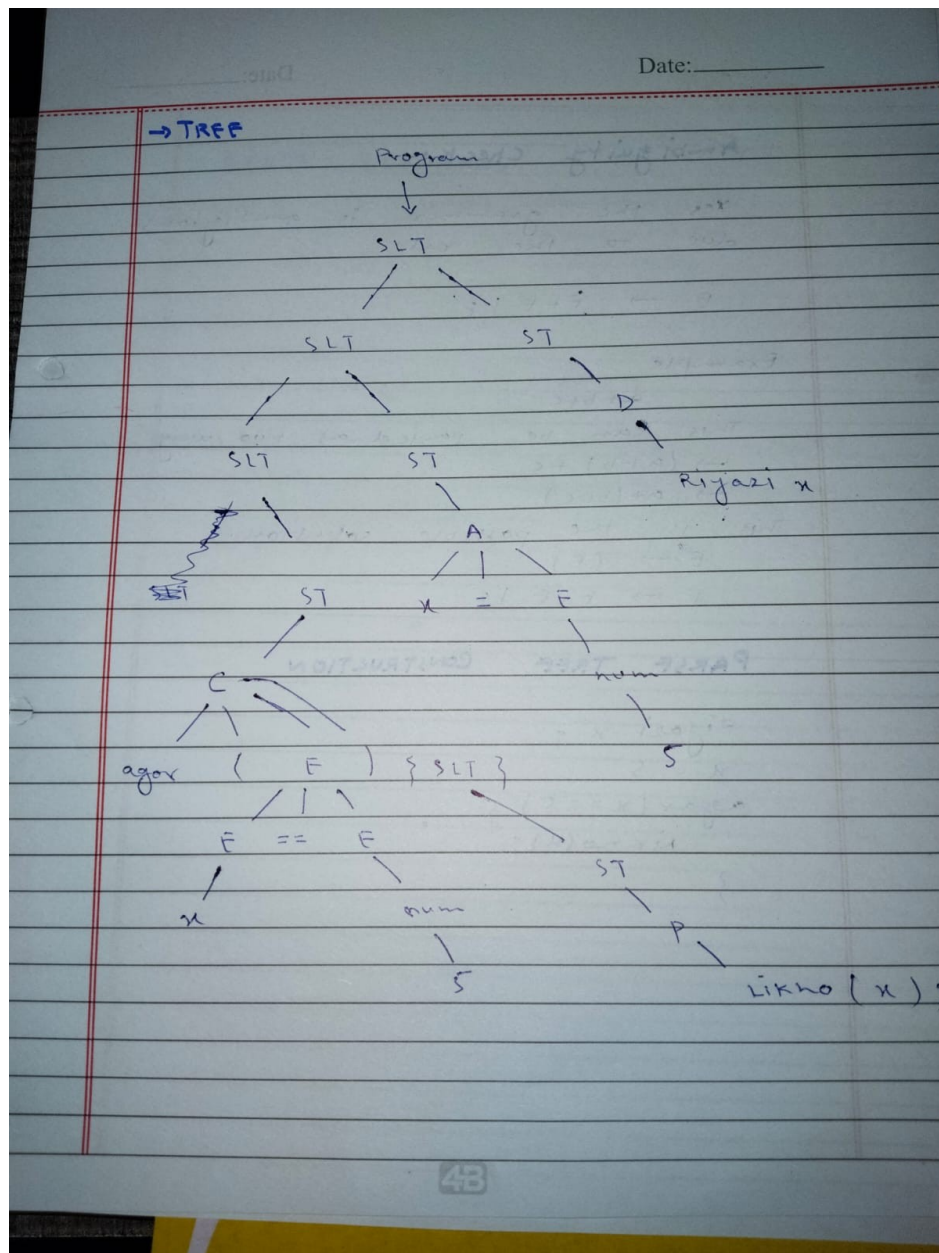
$$x = 5$$

agar (x == 5) {

nikho(x);

}

Parse Tree



Error Handling

Date: _____

ERROR SCENARIOS

(2) `x 5 ;`

(2) `agar x=5 { warn }`

Line 1 → missed the assignment operator
it should be `x=5 ;`

Line 2 → missed the punctuation and using wrong syntax
it should be :

`agar ( x==5 ) { x=1 ; } warn { x=0 ; }`

Phase 01 to Phase 02 Integration Explanation

- Phase 01 scanner identifies tokens using Flex
- Tokens are passed to Phase 02 parser using Bison
- %token declarations in parser.y exactly match scanner.l
- Parser validates syntax only, no semantic analysis
- Successful parse prints confirmation message

Repository Link:

<https://github.com/Asaad-108/Paklang>